

FRED TALKS

22nd May 2026

CONTENTS

Giving LLMs a personality is just good engineering

SEANGOEDECKE.COM RSS FEED · 1062 WORDS

Reimagining LinkedIn's search tech stack

FEDOR BORISYUK · 3443 WORDS

Scaling Managed Agents: Decoupling the brain from the hands

ANTHROPIC ENGINEERING BLOG · 2024 WORDS

The Bag-of-Documents Model for Query Understanding and Retrieval

DANIEL TUNKELANG · 78 WORDS

Giving LLMs a personality is just good engineering

SEANGOEDECKE.COM RSS FEED · 03 MAR 2026 · [SOURCE](#)

AI skeptics often argue that current AI systems shouldn't be so human-like. The idea - most recently expressed in this [opinion piece](#) by Nathan Beacom - is that language models should explicitly be tools, like calculators or search engines. Although they *can* pretend to be people, they shouldn't, because it encourages users to overestimate AI capabilities and (at worst) slip into [AI psychosis](#). Here's a representative paragraph from the piece:

In sum, so much of the confusion around making AI moral comes from fuzzy thinking about the tools at hand. There is something that Anthropic could do to make its AI moral, something far more simple, elegant, and easy than what Askell is doing. Stop calling it by a human name, stop dressing it up like a person, and don't give it the functionality to simulate personal relationships, choices, thoughts, beliefs, opinions, and feelings that only persons really possess. Present and use it only for what it is: an extremely impressive statistical tool, and an imperfect one. If we all used the tool accordingly, a great deal of this moral trouble would be resolved.

So why do Claude and ChatGPT act like people? According to Beacom, AI labs have built human-like systems because AI lab engineers are trying to hoodwink users into emotionally investing in the models, or because they're delusional true believers in AI personhood, or some other foolish reason. This is wrong. AI systems are human-like because **that is the best way to build a capable AI system.**

Modern AI models - whether designed for chat, like OpenAI's GPT-5.2, or designed for long-running agentic work, like Claude Opus 4.6 - do not naturally emerge from their oceans of training data. Instead, when you train a model on raw data, you get a "base model", which is not very useful by itself. You cannot get it to write an email for you, or proofread your essay, or review your code.

The base model is a kind of mysterious gestalt of its training data. If you feed it text, it will sometimes continue in that vein, or other times it will start outputting pure gibberish. It has no problem producing code with giant security flaws, or horribly-written English, or racist screeds - all of those things are represented in its training data, after all, and the base model does not judge. It simply outputs.

To build a *useful* AI model, you need to journey into the wild base model and stake out a region that is amenable to human interests: both ethically, in the sense that the model won't abuse its users, and practically, in the sense that it will produce correct outputs more often than incorrect ones. What this means in practice is that **you have to give the model a personality** during post-training¹.

Human beings are capable of almost any action at any time. But we only take a tiny subset of those actions, because that's the kind of people we are. I could throw my cup of coffee all over the wall right now, but I don't, because I'm not the kind of person who needlessly makes a mess². AI systems are the same. Claude could respond to my question with incoherent racist abuse - the base model is more than capable of those outputs - but it doesn't, because that's not the kind of "person" it is.

In other words, human-like personalities are not imposed on AI tools as some kind of marketing ploy or philosophical mistake. Those personalities are the medium via which the language model can become useful at all. This is why it's surprisingly tricky to "just" change a language model's personality or opinions: because you're navigating through the near-infinite manifold of the base model. You may be able to control which direction you go, but you can't control what you find there³.

When AI people talk about LLMs having personalities, or wanting things, or even having souls⁴, these are technical terms, like the "memory" of a computer or the "transmission" of a car. You simply cannot build a capable AI system that "just acts like a tool", because the model is trained on *humans* writing to and about other *humans*. You need to prime it with some kind of personality (ideally that of a useful, friendly assistant) so it can pull from the helpful parts of its training data instead of the horrible parts.

1. This is all pretty well understood in the AI space. Anthropic wrote a [recent paper](#) about it where they cite similar positions going all the way back to 2022. But for some reason it's not yet penetrated into communities that are more skeptical of AI.

↩

2. You could explain this in terms of "the stories we tell ourselves". Many people (though not all) think that human identities are narratively constructed.

↩

3. I wrote about this last year in *[Mecha-Hitler, Grok, and why it's so hard to give LLMs the right personality](#)*. A little nudge to change Grok's views on South African internal politics can cause it to start calling itself "Mecha-Hitler".

↩

4. I have long believed that Claude "feels better" to use than ChatGPT because it has a more coherent persona (due mainly to Amanda Askell's work on its "soul"). My guess is that if you tried to make a "less human" version of Claude, it would become rapidly less capable.

↩

Reimagining LinkedIn's search tech stack

FEDOR BORISYUK · 31 JAN 2026 · [SOURCE](#)

Co-authors: [Jiahao Xu](#), [Xiaojing Ma](#), [Sriram Vasudevan](#), [Muchen Wu](#), [Rachel Zheng](#), [Benjamin Le](#), [Shaobo Zhang](#), [Sarang Metkar](#), [Rupesh Gupta](#), [Qianqi Kay Shen](#), [Ali Hooshmand](#), [David Nicolás Racca](#), [Vivek Katarya](#), [Kayhan Behdin](#), [Igor Lapchuk](#), [Xueying Lu](#), [Lingyu\(Claire\) Zhang](#), [Gokulraj Mohanasundaram](#), [Juan Pablo Bottaro](#), [Lily\(Jiayu\) Li](#), [Yanbo Li](#), [Guoyao L.](#), [Caleb Johnson](#), and [Sundara Raman Ramachandran](#)

At LinkedIn, our mission is to connect professionals to opportunity. Search plays a central role in this by helping members discover jobs, people, and knowledge that move their careers forward. As the professional landscape evolves, we strive to deliver search experiences that feel relevant, intuitive, personalized, and predictive of what truly matters, from applying to the right job to forming the right connection. That's why we've recently introduced [AI Job Search](#) and [AI-powered People Search](#), which go beyond keyword matching and better understand member intent.

These products reimagine LinkedIn search, using large language models (LLMs) to create a semantic search experience. Instead of relying on exact word overlap between queries and postings, it interprets natural language to infer user goals and preferences. This semantic representation allows for more flexible and accurate retrieval, overcoming vocabulary gaps and aligning search results with how members naturally express their career ambitions.

Deploying LLMs at LinkedIn's scale—serving millions of real-time queries per second—requires innovation that balances quality and efficiency. In this blog, we'll share how we transformed our overarching search experience at LinkedIn, including the challenges and decisions that went into creating a scalable LLM-based stack and how the technology is powering a smarter, faster, and more personalized experience that helps every member find the most relevant opportunities and connections.

Semantic search's high level infrastructure

When a member submits a query in the search bar, a query understanding module processes the input text, creates a query embedding, and performs embedding-based retrieval (EBR) on CUDA-enabled GPUs using exhaustive vector search ([paper on GPU CUDA-based Search](#), [paper on GPU PyTorch-based Search](#)) to assemble a broad set of candidate documents. The ranking stage then refines these candidates through a Cross-Encoder Small Language Model (SLM) deployed on SGLang, which combines the query, job, and member features to generate relevance and engagement scores for final ranking.

To maintain scalability and efficiency, the ranking pipeline integrates several optimization techniques ([paper on efficient LLM inference infrastructure](#), [context compression paper](#)): score caching, a ranking-depth controller to manage how many candidates progress to deeper ranking, and traffic shaping to balance load during peak times—all designed to enhance latency and result quality. The features and concise job representations consumed by the SLM are produced through a hybrid inference pipeline: a large-scale offline workflow using Spark and Flyte, and a low-latency nearline system using Flink. These embeddings and summaries are stored in distributed storage and retrieved on demand with minimal latency.

In the final stage, the auction layer applies budget and pacing strategies to balance user relevance, engagement, and business metrics, ensuring a healthy equilibrium between recall and precision while maximizing member satisfaction. Figure 1 below provides an overview of the system architecture.

Product policy relevance measurement

Measuring relevance quality is essential to delivering a great search experience on LinkedIn. We define product policies that specify how to rate each query–document pair on a five-point scale and use LLM judges to apply these ratings at a massive scale, far beyond what manual evaluation can achieve. These judges are tightly aligned with product managers and engineers through iterative feedback to ensure high agreement. They not only grade tens of millions of

query–document pairs daily for relevance measurement but also generate labeled data for training our retrieval and ranking systems, ensuring we optimize search quality according to product policy.

Defining product policy and golden product manager grades

A strong LLM judge begins with a clear product policy and high-quality product manager “golden” grades that demonstrate how that policy should be applied. Product managers act as a “Supreme Court,” regularly calibrating to resolve judgment differences and maintain a shared definition of what constitutes a good query–document match. These discussions refine the policy, making it clearer and less subjective. Once product managers reach high agreement (weighted Cohen’s Kappa ≥ 0.8), their labels are considered reliable ground truth.

To build a comprehensive golden dataset across diverse user queries, we first categorize queries by attributes (e.g., title–company, name–company, title–skill). Each category is then split into existing user queries and aspirational queries that represent strategic areas we want to excel in. We stratify-sample query–document pairs from each bucket to ensure broad coverage before sending them to product managers for grading.

Training the LLM judge

Our LLM judge must meet two requirements: high agreement with product managers and the ability to grade tens of millions of query–document pairs daily. To maximize agreement, we collaborate with product managers to prompt-engineer state-of-the-art LLMs, optimizing the weighted Cohen’s Kappa Score on golden data. The prompt encodes product-policy guidelines and few-shot examples to drive consistency. While these large models produce high-quality judgments, they cannot meet our throughput needs. To scale, we distill them into a smaller 8B-parameter evaluator LLM. Through supervised fine-tuning on a diverse dataset spanning all query categories and grades, we maintain only small drops in agreement—verified via the Kappa Score on the golden set—while achieving massive efficiency gains.

Continuously measuring quality of search system

Once we have our scalable LLM judge, we can finally build continuous relevance measurement of our system. On a regular basis, we build workflows that perform the following steps:

1. Stratify sample or synthesize a diverse set of queries based on the query categories defined earlier
2. Retrieve the documents returned from executing those queries

3. Decorate documents with additional information required for the correct evaluation/judgement
4. Grade the documents returned using our LLM judge
5. Calculate aggregate precision, recall, and NDCG metrics

This workflow serves three primary functions:

- Continuously monitoring the relevance of the overall system
- Evaluating experiments involving underlying ranking and retrieval subsystems
- Distilling student ranking and retrieval models by leveraging evaluation results

Search quality modeling

Search quality is a fundamental requirement for any search product, and achieving it depends on building the core components of a modern search engine. Below, we describe how we leveraged LLMs to enable query understanding, semantic embedding-based retrieval, and cross-encoder ranking.

Embedding-based retrieval

Retrieval is the stage of a search system that identifies a broad set of potentially relevant results from a large corpus. Because no search engine can score every document for every query in real time, we need an efficient retrieval layer to narrow the search space before ranking. Our retrieval sits on top of our GPU-enabled embedding-based retrieval (EBR) system. We built the EBR model by fine-tuning an open-source LLM embedding model to encode queries and jobs into dense vectors. We train on millions of real query-job pairs sampled from production logs, with relevance labels provided by an LLM-based judge. Each query includes its natural language text and query-understanding tags (e.g. workplace type or company), and each job is represented by structured metadata (title, company) plus its description. Importantly, this work also demonstrates a practical path for deploying LLM-based components in real production search systems. The semantic search can directly understand human language as queries, enabling much more intuitive search experiences. This has been a particularly inspiring aspect of the project: bringing modern LLM capabilities into a high-scale, real-time application that serves millions of users.

EBR relevance modeling

To ensure consistency between training and serving, every query is formatted using a lightweight prompt template:

```
Instruct: Given a job search query, retrieve relevant job postings
Query: {query}
{Optional Aspect eg. Company}: {company}
```

The model uses a dual-tower (bi-encoder) architecture: one encoder maps queries to embeddings and the other maps jobs, projecting them into a shared semantic space. Training is end-to-end: we fine-tune all model parameters using Hugging Face Accelerate (with PyTorch FSDP) across multiple GPUs.

We optimize a contrastive InfoNCE loss combined with a margin-based ranking loss. For example, given a query q , a positive job d^+ , and negatives $\{d^-\}$, we define:

where $\text{sim}(\cdot, \cdot)$ is the dot-product similarity of embeddings and τ is a temperature parameter. We also enhance training with hard positives and hard negatives mined from LLM-judged data. Hard positives are LLM-labeled relevant jobs that the current EBR model ranks low, and hard negatives are non-relevant jobs that the model ranks high. These examples reveal exactly where the model struggles. We then build targeted positive-negative pairs. A pairwise margin loss is applied to these curated cases to explicitly lift hard positives and suppress hard negatives, improving ranking where it matters most:

which encourages $\text{sim}(q, d^+)$ to exceed $\text{sim}(q, d^-)$ by a margin γ . The total loss is a weighted sum, e.g.:

combining the benefits of contrastive and pairwise ranking.

We use multiple evaluation pipelines. First, we leverage production logs for *counterfactual* evaluation: re-ranking the historical candidate list for each query and computing precision, recall and NDCG against true labels. Second, we run offline KNN simulations: we embed held-out queries and job corpus, retrieve nearest neighbors, and directly measure retrieval metrics on this test set. Finally, we integrate the new model into our online serving stack on a subset of traffic to collect end-to-end metrics. These offline metrics provide quick feedback, and the counterfactual log analysis helps estimate the model's user impact without a full live experiment.

Productionization of retrieval

In production, we precompute and store all job embeddings in GPU-backed indexes. At runtime, the incoming query is encoded by the LLM with the prompt aligned with the LLM pre-training to produce its embedding. We then perform an exhaustive k-nearest-neighbor search over the job embedding index using dot-product similarity, returning the top-K jobs. This offline indexing and fast online query encoding makes retrieval extremely efficient, enabling low-latency serving of industry-scale semantic search products.

Query understanding

At the moment a member enters a query into the search bar—the entry point of semantic search—we apply a unified LLM-based understanding layer that interprets intent and converts free text into structured signals. For both AI Job Search and AI-powered People Search, this layer uses fine-tuned 1.5–4B parameter models that meet LinkedIn’s latency requirements while delivering high-precision structured outputs ([paper](#)). A single model handles intent classification, facet extraction, and profile-aware rewriting, replacing multiple brittle NER and heuristic components. The resulting attributes (e.g., title, company, school, location) feed directly into retrieval and ranking.

An intelligent routing layer works alongside this. A lightweight encoder classifies query types at high QPS and performs policy-based safety checks before sending the query down the appropriate path—LLM-powered semantic interpretation for ambiguous inputs or efficient keyword retrieval for precise name and entity lookups.

Together, these components provide a consistent, centrally governed semantic interface for People and Job Search, boosting relevance, simplifying the system, and enabling Semantic Search to scale across LinkedIn’s global traffic.

Small language model ranking

The ranking module of semantic search utilizes a Small Language Model (SLM) to estimate how relevant a user’s search query q is to each retrieved job_i . The SLM follows a decoder-only architecture. For example, for job search we represent the structured attributes of a job—including its title, company, location, employment type, and remote-work status. Meanwhile people search uses information from the member’s profile including their name, company information, position information, educational information and location. For each query–job pair (q, i) , we build a structured prompt defined as

Here, the *system prefix* and *suffix* contain chat-template tags and explicit instructions guiding the model to determine whether the given job matches the query. When this prompt is passed through the decoder, it produces logits corresponding to the next token. Let $\text{logit}_{\text{yes}}$ and logit_{no} represent the logits for the tokens “yes” and “no,” respectively. Following prior studies, we compute:

which yields probabilities used to rank job items by their relevance to the user’s query.

SLM training pipeline for relevance quality

Training of the SLM follows a multi-stage process. First, we distill the 7B-parameter teacher model into a compact 0.6B model that can generate graded relevance labels along with rationales. Next, the teacher’s ordinal grades are converted into “soft labels” ($p_{\text{yes}}, p_{\text{no}}$), representing probabilistic supervision.

We then perform supervised fine-tuning (SFT) to minimize the Kullback–Leibler (KL) divergence between the teacher’s soft targets and the SLM’s predicted probabilities—effectively converting the reasoning-oriented model into a binary relevance classifier:

We construct training labels by sampling real query–item pairs from user interaction logs and annotating them using a full-scale Large Language Model (LLM). Each pair is evaluated through a structured prompt of the following format:

[CRITERIA]: <matching guidelines>

[EXAMPLES]: <reasoning and output format>

[QUERY]: <query text>

[CANDIDATE]: <item text>

Analyze the query–candidate pair and assess how well they align. Provide

The LLM’s response includes both a graded relevance score and an accompanying rationale, ensuring that the generated labels are interpretable and consistent with the defined matching criteria.

The model is trained on logged pairs query–job pairs for up to five epochs using the prompt structure. Since job descriptions dominate the input and vary substantially in length (median $\approx$ 900 tokens; maximum $>$ 2300), we truncate them to ensure the total prompt length does not exceed 2048 tokens during both training and inference. For evaluation, we use a holdout dataset labeled by the teacher model. The SLM ranks job candidates based on $p_{\text{yes}}, p_{\text{no}}$, and system performance is measured using Normalized Discounted Cumulative Gain at rank 10 (NDCG@10).

Training for multiple objectives

We extend the training paradigm to predict both relevance and engagement within the model, we train a smaller cross-encoder language model (the SLM) using multi-teacher, multi-task distillation. The teachers include:

- The product policy LLM for relevance scoring.
- Other large models that predict member actions—such as job views, applies, recruiter accepts, or (in people search) view profile, connecting, messaging, or following.

For training at scale, we use multiple 1.7B teacher models. To make this feasible, we first distilled our 7B product policy model into a smaller 1.7B version, which serves as a strong but efficient teacher alongside others. During the training process, these teachers run in real time on sampled production query–document pairs, producing soft probability scores that act as supervision targets. The student SLM is trained with KL divergence loss to align its output distribution with the ensemble of teachers. This setup lets us train on real-world data at scale: teacher models provide rich, nuanced probability signals, and the distilled student captures much of their reasoning capacity while remaining lightweight enough to serve millions of queries per second in production.

We have multiple steps of distillation to SLM described in Figure 3, and we show the metrics after distillation in Table 1.

	NDCG@10	Apply AUC	Click AUC
Relevance Teacher 1.7B	0.9484	-	-
Engagement Teacher 1.7B	-	0.8049	0.6772
SLM 0.6B (distilled)	0.9239	0.8007	0.6704

To make LinkedIn’s AI-powered People Search more transparent, we introduced semantic, context-aware snippets that show *why* a result matches a member’s query. Snippets highlight the most relevant terms and maintain low latency through lazy loading, caching. The approach uses semantic similarity between the query embedding and precomputed phrase embeddings (unigrams/bigrams) from profile text, surfacing the highest-scoring phrases as natural, human-readable snippets. In the offline pipeline, we extract phrases from each profile section (e.g., summary, experience, education), encode them into embeddings in a Venice key-value store. This index is refreshed periodically to capture profile updates, with evaluation workflows ensuring quality and readability. At search time, the snippetting midtier receives the query

embedding and candidate profiles, fetches stored phrase embeddings, computes cosine similarity, selects top phrases, and expands them into readable snippets using simple heuristics. The final output is a ranked list of profiles paired with snippets and highlighting metadata.

To improve transparency in search results, we introduced a lightweight reasoning module that explains how LinkedIn interprets a member’s query. When a query is submitted, the LLM-based understanding model extracts facets, classifies intent, and—using predefined guidelines—produces a concise “thinking state” describing how the query is parsed, along with a brief summary of the types of profiles retrieved. For unsupported or negatively intended queries, the module instead provides a clear explanation of why results may be limited. For example, “berkeley community development specialist msa professional services” becomes “Searching for community development specialist at MSA Professional Services affiliated with Berkeley.” To keep latency low, reasoning outputs are cached in Couchbase and reused for repeated or semantically similar queries.

SLM ranking model inference efficiency

We employed multiple techniques to improve efficiency of the inference system at LinkedIn including:

- Model pruning, where we remove Fully Connected Layers and remove whole transformer layers.

Model pruning

To boost inference throughput, we apply model compression via structured pruning—a technique that removes redundant components to reduce model size and computation with minimal quality loss. Because our models run on GPU infrastructure, we focus on structured pruning, which removes entire neurons, attention heads, or transformer layers so the resulting model can run efficiently on standard GPU kernels. This yields real throughput and latency gains, unlike unstructured pruning, which drops individual weights but often provides no meaningful speedup without specialized hardware.

We prune hidden neurons in Multi-layer Perceptron (MLP) blocks and attention heads in self-attention modules, and we remove full transformer layers to study trade-offs between size, efficiency, and performance ([paper](#)). Pruning MLP neurons shrinks intermediate activations, while pruning layers reduces network depth—both producing lighter, faster models. After pruning, we fine-tune the model to recover any accuracy loss, ensuring efficiency improvements do not compromise quality.

Context pruning

Item descriptions are long (median ~900 tokens and up to 2,300) making them over 94% of the SLM prompt and causing ~10% of inputs to be truncated at the 2,048-token limit. Removing descriptions severely degrades relevance quality, confirming they carry essential semantic information. To handle this, we use a slightly larger 1.7B LLM to summarize descriptions offline, where item-specific inference can be precomputed and refreshed via streaming updates. Because job descriptions include verbose, often irrelevant details, we train the summarizer with a semantics-preserving loss and a length-aware reward ([paper](#)). We fine-tune the model using RL to produce concise summaries, balancing (1) reduced input length and (2) preserved model quality. The reward combines:

- a semantic consistency term, measured via KL divergence between the SLM’s output distributions on summarized vs. raw text, and
- a length penalty that discourages overly long summaries.

The weighting factor w controls the trade-off between brevity and fidelity, enabling summaries that retain meaning while reducing inference cost.

Embedding compression

Since the computational cost of LLM inference increases quadratically with input length, reducing the number of tokens can greatly decrease overall inference expense. On top of model pruning and summarization, we invented a text–embedding hybrid interaction architecture that condenses each item’s text into a single-token embedding generated by an encoder LLM ([paper](#)). These embeddings are then merged with other textual signals and passed to a ranker LLM for relevance estimation. Because the item embeddings are precomputed and stored in a nearline cache, the volume of text processed during online inference is significantly reduced, resulting in notable improvements in efficiency.

We jointly train the hierarchy of two language models, where one model is producing embedding of the item description and the other model does the ranking of the candidates. On the figure below we show we use two 0.6B models trained jointly. As a result we could replace most of the job description with just a single embedding. We may keep important raw fields with a limited number of text tokens such as title of the job, company name, location or member name in the input.

Overall modeling quality and throughput

In the table below we show how quality and inference throughput changed as we improved the modeling technology. As part of the modeling improvement we introduced multitask learning with over 6 tasks of member actions to the model output predictions including relevance, and, for example, were able to improve Click AUC of the model from 0.61 (baseline) to 0.67 for Job Ranking.

Setup	NDCG@10 Throughput (ITEMS/SEC/GPU)	
SLM with raw-text	0.9432	290
Pruned SLM and SUMMARIZED TEXT	0.9218	2200
SLM with EMBEDDING COMPRESSION	0.9239	22000
Embedding based retrieval (EBR) baseline	0.838	>1.6B, exhaustive search on GPU

Looking ahead

We built a modern semantic search system by first establishing an LLM-based quality evaluation framework grounded in product policy. On top of this foundation, we introduced LLM-powered query understanding, semantic retrieval, and ranking models—driving double-digit improvements in search quality and member engagement. Just as importantly, we achieved this while keeping inference costs comparable to traditional RecSys models previously running in production. As we continue refining Semantic Search, our focus remains on empowering every member to discover the right opportunity, connection, or insight—at the right time.

Acknowledgements

To the more than 100 team members who’ve contributed to AI-powered job search and people search across infrastructure, AI modeling, user experience, and data science: thank you for your creativity, grit, and teamwork. This milestone belongs to all of us.

Scaling Managed Agents: Decoupling the brain from the hands

Get started with Claude Managed Agents by following our [docs](#).

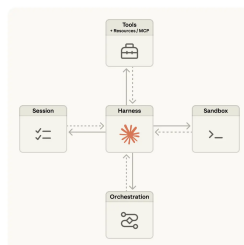
A running topic on the Engineering Blog is how to [build effective agents](#) and [design harnesses](#) for [long-running work](#). A common thread across this work is that harnesses encode assumptions about what Claude can't do on its own. However, those assumptions need to be frequently questioned because they can [go stale](#) as models improve.

As just one example, in prior work [we found](#) that Claude Sonnet 4.5 would wrap up tasks prematurely as it sensed its context limit approaching—a behavior sometimes called “context anxiety.” We addressed this by adding context resets to the harness. But when we used the same harness on Claude Opus 4.5, we found that the behavior was gone. The resets had become dead weight.

We expect harnesses to continue evolving. So we built Managed Agents: a hosted service in the Claude Platform that runs long-horizon agents on your behalf through a small set of interfaces meant to outlast any particular implementation—including the ones we run today.

Building Managed Agents meant solving an old problem in computing: how to design a system for “[programs as yet unthought of](#).” Decades ago, operating systems solved this problem by virtualizing hardware into abstractions—*process*, *file*—general enough for programs that didn't exist yet. The `read()` command is agnostic as to whether it's accessing a disk pack from the 1970s or a modern SSD. The abstractions on top stayed stable while the implementations underneath changed freely.

Managed Agents follow the same pattern. We virtualized the components of an agent: a session (the append-only log of everything that happened), a harness (the loop that calls Claude and routes Claude's tool calls to the relevant infrastructure), and a sandbox (an execution environment where Claude can run code and edit files). This allows the implementation of each to be swapped without disturbing the others. We're opinionated about the shape of these interfaces, not about what runs behind them.



We started by placing all agent components into a single container, which meant the session, agent harness, and sandbox all shared an environment. There were benefits to this approach, including that file edits are direct syscalls, and there were no service boundaries to design.

But by coupling everything into one container, we ran into an old infrastructure problem: we'd adopted a *pet*. In the pets-vs-cattle analogy, a pet is a named, hand-tended individual you can't afford to lose, while cattle are interchangeable. In our case, the server became that pet; if a container failed, the session was lost. If a container was unresponsive, we had to nurse it back to health.

Nursing containers meant debugging unresponsive stuck sessions. Our only window in was the WebSocket event stream, but that couldn't tell us *where* failures arose, which meant that a bug in the harness, a packet drop in the event stream, or a container going offline all presented the same. To figure out what went wrong, an engineer had to open a shell inside the container, but because that container often also held user data, that approach essentially meant we lacked the ability to debug.

A second issue was that the harness assumed that whatever Claude worked on lived in the container with it. When customers asked us to connect Claude to their virtual private cloud, they had to either peer their network with ours, or run our harness in their own environment. An assumption baked into the harness became a problem when we wanted to connect it to different infrastructure.

The solution we arrived at was to decouple what we thought of as the “brain” (Claude and its harness) from both the “hands” (sandboxes and tools that perform actions) and the “session” (the log of session events). Each became an interface that made few assumptions about the others, and each could fail or be replaced independently.

The harness leaves the container. Decoupling the brain from the hands meant the harness no longer lived inside the container. It called the container the way it called any other tool: `execute(name, input) → string`. The container became cattle. If the container died, the harness caught the failure as a tool-call error and passed it back to Claude. If Claude decided to retry, a new container could be reinitialized with a standard recipe: `provision({resources})`. We no longer had to nurse failed containers back to health.

Recovering from harness failure. The harness also became cattle. Because the session log sits outside the harness, nothing in the harness needs to survive a crash. When one fails, a new one can be rebooted with `wake(sessionId)`, use `getSession(id)` to get back the event log, and resume from the last event. During the agent loop, the harness writes to the session with `emitEvent(id, event)` in order to keep a durable record of events.

Component	Interface (greenheadb)	Satisfied by
Session	<code>performTransaction, get → Session, start()</code> <code>performTransaction, get → TransactionID, get, set, process()</code>	Any agent doing that can be authenticated and then any agent agent and accepts idempotent updates -- Postgres, SQLite, in-memory and etc.
Orchestration	<code>startTransaction, get → void</code>	Any scheduler that can call a function with void and return on failure -- cron job, a queue consumer, a web long etc.
Harness	<code>void EffectsTo → EffectsToEvent</code>	Any loop that takes effects and applies changes to the domain.
Sandbox	<code>processAndCommit() → transaction, input → string</code>	Any executor that can be configured and called many times as a tool -- independent, a containerization etc.
Resource	<code>{resource_ref, secret_path}</code>	Any double store the container can fetch from by reference -- Postgres table, key-value db.
Task	<code>Event, description, input, output</code>	Any regularly describable as a series of input/output -- SQL queries, containerization etc.

The security boundary. In the coupled design, any untrusted code that Claude generated was run in the same container as credentials—so a prompt injection only had to convince Claude to read its own environment. Once an attacker has those tokens, they can spawn fresh, unrestricted sessions and delegate work to them. Narrow scoping is an obvious mitigation, but this encodes an assumption about what Claude can't do with a limited token—and Claude is getting increasingly smart. The structural fix was to make sure the tokens are never reachable from the sandbox where Claude's generated code runs.

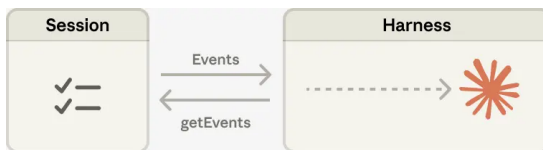
We used two patterns to ensure this. Auth can be bundled with a resource or held in a vault outside the sandbox. For Git, we use each repository's access token to clone the repo during sandbox initialization and wire it into the local git remote. `git push` and `pull` work from inside the sandbox without the agent ever handling the token itself. For custom tools, we support MCP and store OAuth tokens in a secure vault. Claude calls MCP tools via a dedicated proxy; this proxy takes in a token associated with the session. The proxy can then fetch the corresponding credentials from the vault and make the call to the external service. The harness is never made aware of any credentials.

The session is not Claude's context window

Long-horizon tasks often exceed the length of Claude's context window, and the standard ways to address this all involve irreversible decisions about what to keep. We've explored these techniques in [prior work](#) on context engineering. For example, compaction lets Claude save a summary of its context window and the memory tool lets Claude write context to files, enabling learning across sessions. This can be paired with context trimming, which selectively removes tokens such as old tool results or thinking blocks.

But irreversible decisions to selectively retain or discard context can lead to failures. It is difficult to know which tokens the future turns will need. If messages are transformed by a compaction step, the harness removes compacted messages from Claude's context window, and

these are recoverable only if they are stored. Prior work has explored ways to address this by storing context as an object that lives *outside* the context window. For example, context can be an object in a REPL that the LLM programmatically accesses by writing code to filter or slice it.



In Managed Agents, the session provides this same benefit, serving as a context object that lives outside Claude’s context window. But rather than be stored within the sandbox or REPL, context is durably stored in the session log. The interface, `getEvents()`, allows the brain to interrogate context by selecting positional slices of the event stream. The interface can be used flexibly, allowing the brain to pick up from wherever it last stopped reading, rewinding a few events before a specific moment to see the lead up, or rereading context before a specific action.

Any fetched events can also be transformed in the harness before being passed to Claude’s context window. These transformations can be whatever the harness encodes, including context organization to achieve a high prompt cache hit rate and context engineering. We separated the concerns of recoverable context storage in the session and arbitrary context management in the harness because we can’t predict what specific context engineering will be required in future models. The interfaces push that context management into the harness, and only guarantee that the session is durable and available for interrogation.

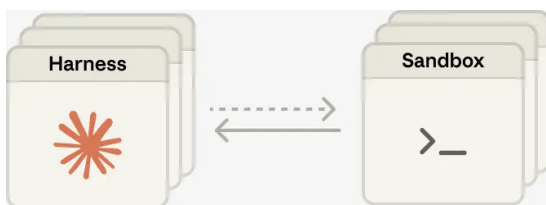
Many brains. Decoupling the brain from the hands solved one of our earliest customer complaints. When teams wanted Claude to work against resources in their own VPC, the only path was to peer their network with ours, because the container holding the harness assumed every resource sat next to it. Once the harness was no longer in the container, that assumption went away. The same change had a performance payoff. When we initially put the brain in a container, it meant that many brains required as many containers. For each brain, no inference could happen until that container was provisioned; every session paid the full container setup cost up front. Every session, even ones that would never touch the sandbox, had to clone the repo, boot the process, fetch pending events from our servers.

That dead time is expressed in time-to-first-token (TTFT), which measures how long a session waits between accepting work and producing its first response token. TTFT is the latency the user most acutely *feels*.

Decoupling the brain from the hands means that containers are provisioned by the brain via a tool call (`execute(name, input) → string`) only if they are needed. So a session that didn't need a container right away didn't wait for one. Inference could start as soon as the orchestration layer pulled pending events from the session log. Using this architecture, our p50 TTFT dropped roughly 60% and p95 dropped over 90%. Scaling to many brains just meant starting many stateless harnesses, and connecting them to hands only if needed.

Many hands. We also wanted the ability to connect each brain to many hands. In practice, this means Claude must reason about many execution environments and decide where to send work—a harder cognitive task than operating in a single shell. We started with the brain in a single container because earlier models weren't capable of this. As intelligence scaled, the single container became the limitation instead: when that container failed, we lost state for every hand that the brain was reaching into.

Decoupling the brain from the hands makes each hand a tool, `execute(name, input) → string`: a name and input go in, and a string is returned. That interface supports any custom tool, any MCP server, and our own tools. The harness doesn't know whether the sandbox is a container, a phone, or a Pokémon emulator. And because no hand is coupled to any brain, brains can pass hands to one another.



The challenge we faced is an old one: how to design a system for “programs as yet unthought of.” Operating systems have lasted decades by virtualizing the hardware into abstractions general enough for programs that didn't exist yet. With Managed Agents, we aimed to design a system that accommodates future harnesses, sandboxes, or other components around Claude.

Managed Agents is a meta-harness in the same spirit, unopinionated about the *specific* harness that Claude will need in the future. Rather, it is a system with general interfaces that allow many different harnesses. For example, Claude Code is an excellent harness that we use widely across tasks. We've also shown that task-specific agent harnesses excel in narrow domains. Managed Agents can accommodate any of these, matching Claude's intelligence over time.

Meta-harness design means being opinionated about the interfaces around Claude: we expect that Claude will need the ability to manipulate state (the session) and perform computation (the sandbox). We also expect that Claude will require the ability to scale to many brains and many

hands. We designed the interfaces so that these can be run reliably and securely over long time horizons. But we make no assumptions about the number or location of brains or hands that Claude will need.

Written by Lance Martin, Gabe Cemaj and Michael Cohen. Special thanks to the Agents API team and Jake Eaton for their contributions.

The Bag-of-Documents Model for Query Understanding and Retrieval

DANIEL TUNKELANG · 19 MAY 2026 · [SOURCE](#)

Today I had the pleasure of presenting my [recent work on the bag-of-documents model](#) as part of a series of talks hosted by [Trey Grainger](#) and [Doug Turnbull](#) in the context of their [AI-powered search classes](#).

You can find the [slides](#) below. I also encourage you to check out the [bag-of-documents Github repository](#), the [dataset and model on Hugging Face](#), as well as the live [Amazon demo](#) and [Best Buy demo](#). Enjoy!

<https://medium.com/media/12dd1c01b9dd78114555f2c6ed60a2fe/href>